PROJECT1: Where in the World?

Tejvir Thomas · UMID 00619123 · September 28, 2026

Prompt to Elephant

Using my submitted HW1 “Where in the World?” design, update the design for implementation of Project 1. The instructor told me to proceed using the General Feedback because individualized feedback may not arrive before the deadline. Address every General Feedback item exactly: keep AGENTS.md current and use current Swift, SwiftUI, Python, Starlette, and Granian patterns; avoid enterprise-scale abstractions and robustness beyond the tutorials; make the mystery city fixed throughout each game; and support LLMs both with and without reasoning/thinking output.

Preserve the tutorial architecture, naming, OpenAI-compatible request data, /llmchat protocol, and the existing SQLite turns schema. Design system-only, user-only, and combined prompts so one Send tap produces exactly one POST and omits empty roles. Add /llmclear by reusing the existing OpenAIRequest-compatible shape. Include a second game that reuses all infrastructure. Serialize Send, Clear, initialization, and game switching. Ensure the project frontend works with mada.eecs.umich.edu and the private backend, and tutorial frontends continue working with the private backend. Produce a concise design summary with no code, suitable to give verbatim to a brand-new Goldfish reviewer.

Feedback context

No individualized grading comments were available. Professor Jamin replied that work should proceed while addressing all General Feedback in the Project 1 specification, because most individualized feedback falls within those items.

Elephant Design Summary (No Code)

The project remains a tutorial-sized SwiftUI and Python/Starlette extension of llmChat. The single screen shows the “Where in the World?” title, a City/Landmark game selector, the chronological prompt-completion timeline, separate optional system and user editors, Send, and Clear. A single view model owns those values plus one request phase shared by launch initialization, Send, Clear, and game switching. This prevents overlapping mutations without introducing enterprise concurrency machinery.

At launch, the client sends one system message containing the selected game rules. The city rules require the model to select one mystery city and keep it fixed until the user wins or gives up, reveal hints without naming it, and on success provide visible decimal coordinates and offer another round. The Landmark game follows the same lifecycle and infrastructure with one fixed landmark. Both prompts work whether reasoning tokens are present or absent.

Send snapshots the two editors and creates messages in system-then-user order, omitting either role when blank. Both drafts remain until one Send tap creates exactly one llmChat request. The existing OpenAI-compatible fields and llmChat behavior are unchanged. Visible completion tokens stream into one assistant timeline item; optional reasoning is parsed without exposing hidden thoughts that could reveal the mystery. Drafts clear only after success. Failures preserve retryable input and show an alert; an SSE error event also inserts the required prefixed message into the assistant bubble.

The backend keeps the existing turns schema. Each new turn stores the unchanged newly submitted message array as bytes in the existing prompt BLOB, allowing system and user roles to be reconstructed. Existing tutorial rows remain compatible by using SQLite storage class: BLOB rows decode as validated message arrays, while legacy TEXT rows become user messages. Context is isolated by appID and ordered by turnID. Completion and optional reasoning accumulate during streaming and persist on completion.

The new llmclear endpoint decodes the same request shape, uses appID to delete matching turns transactionally, and returns HTTP success without adding a DTO or changing the database. User Clear leaves server history, the timeline, and both prompt editors genuinely empty. Switching games performs a serialized clear then initialization, preventing old rules or hidden answers from leaking into the new game.

Tests cover system-only, user-only, combined and empty input; both games; fixed-answer lifecycle; thinking and non-thinking streams; appID isolation; legacy and structured persistence; UTF-8 chunk boundaries; errors; clear behavior; rapid taps; clean game switching; and interoperability among the project client, mada, private backend, and tutorial clients.

Prompt to Goldfish (Brand-New Session)

Act as a senior engineer reviewing a course project design with no prior conversation context. Do not write code. Identify correctness and specification risks, architecture or naming problems, missing edge cases, and tests that should be added. Pay special attention to fixed game state, shared appID context, system/user message persistence without a schema change, /llmclear compatibility, streaming failures, and avoiding over-engineering. End with a concise “Goldfish Design Critique” suitable for verbatim inclusion in a student PDF.

[The Elephant Design Summary on the preceding page was then supplied verbatim.]

Review boundary

The Goldfish received no earlier project conversation or repository history. Its only design input was the Elephant summary, satisfying the fresh-context review requirement.

Goldfish Design Critique (No Code)

The design is appropriately small and preserves the required request and database schemas, but its game lifecycle is not yet fully specified. Most importantly, selecting a different game merely sends another system prompt into history keyed by the same appID; this can mix old rules and hidden answers with the new game. Game switching, startup initialization, Send, and the Clear-then-reinitialize sequence should share one serialized state machine, with a defined clean round boundary and recovery when initialization fails. The JSON-in-BLOB compatibility strategy is viable only if structured rows are unambiguously validated, legacy JSON-looking user text remains readable, and reconstruction has deterministic ordering without duplicating client history. The design should also define error and cancellation behavior for drafts, partial assistant placeholders, missing SSE completion, and edits made during streaming. Finally, fixed answers and accurate coordinates are prompt-level expectations rather than deterministic guarantees, so protocol and persistence tests should use mocked model output and avoid claiming that prompt wording alone enforces game state.

Resolution summary

Game selection now clears shared context before new initialization.
All mutations share one request phase.
SQLite BLOB versus TEXT unambiguously separates structured and legacy rows.
Draft/error/partial-stream behavior and deterministic turn ordering are explicit.
Mocked protocol tests are separated from live model-behavior checks.

Final Updated Design

Responsibilities

Component	Responsibility
ContentView	Renders required controls and timeline; delegates actions.
ChatViewModel	Owns selected game, drafts, turns, alerts, initialization guard, and idle/initializing/sending/clearing phase.
Chat	One visible system, user, or assistant timeline entry.
Rein	Creates unchanged requests, calls llmchat/llmclear, validates HTTP, parses official SSE events, and streams visible content while safely handling optional reasoning.
main.py	Registers the existing routes plus llmclear.
handlers.py	Validates requests, stores/reconstructs appID context, routes runners, accumulates streams, persists completions, and clears context.

Lifecycle

launch → initialize selected fixed-mystery rules → stream hints → idle
idle + Send → one request containing nonempty system/user drafts → stream → persist → idle
idle + Clear → delete appID history → clear local timeline and both editors → idle
idle + game switch → clear old appID → initialize new fixed game → idle
request failure → preserve retryable drafts, show an alert, return idle; SSE error event → also write the required prefixed assistant error bubble

Protocol and persistence invariants

OpenAIRequest-compatible fields remain appID, model, messages, and stream.
llmchat is unchanged at the wire boundary; clear reuses the same shape.
No table, column, trigger, or index changes.
Only newly submitted messages are sent; the server owns historical context.
turnID—not timestamps—defines order.
The mystery changes only at an explicit round boundary.

Final Design: Verification and Rationale

Required verification

System-only, user-only, and combined drafts; blank roles omitted; exactly one POST.
City and Landmark initialization, wrong guesses, give-up, success coordinates, and play-again.
Reasoning/content streams, non-thinking streams, SSE errors, malformed/missing completion, split Unicode.
appID isolation; structured BLOB and legacy TEXT; completion/reasoning/timestamps; clear affects only target app.
Project frontend with mada and private backend; tutorial frontend with private backend.
Clean clone, backend install/start, frontend open/build/run, required root files and tags.

Engineering justification

The design deliberately retains the tutorial's small number of files and direct data flow. One request phase is sufficient because the UI explicitly disallows concurrent Send/Clear/switch operations. SQLite WAL and indexed appID/turnID access remain as taught. The second game is data-driven through its initialization rules and therefore proves reuse without adding a generic framework.

Honor Code Pledge

"I [HAVE] received aid from an LLM or automated coding assistant on this submitted work. If I received aid, the specific tool used was (specify model and version): [OpenAI Codex, GPT-5]. Regardless of assistance, I fully understand and can defend all code and design choices within this submission, and I have not concealed any violations of the Honor Code."

Signed: Tejvir Thomas UMID: 00619123 Date: September 28, 2026

Estimated HW1 + Project 1 AI token cost: unavailable from the client interface; no fabricated monetary estimate is provided.